

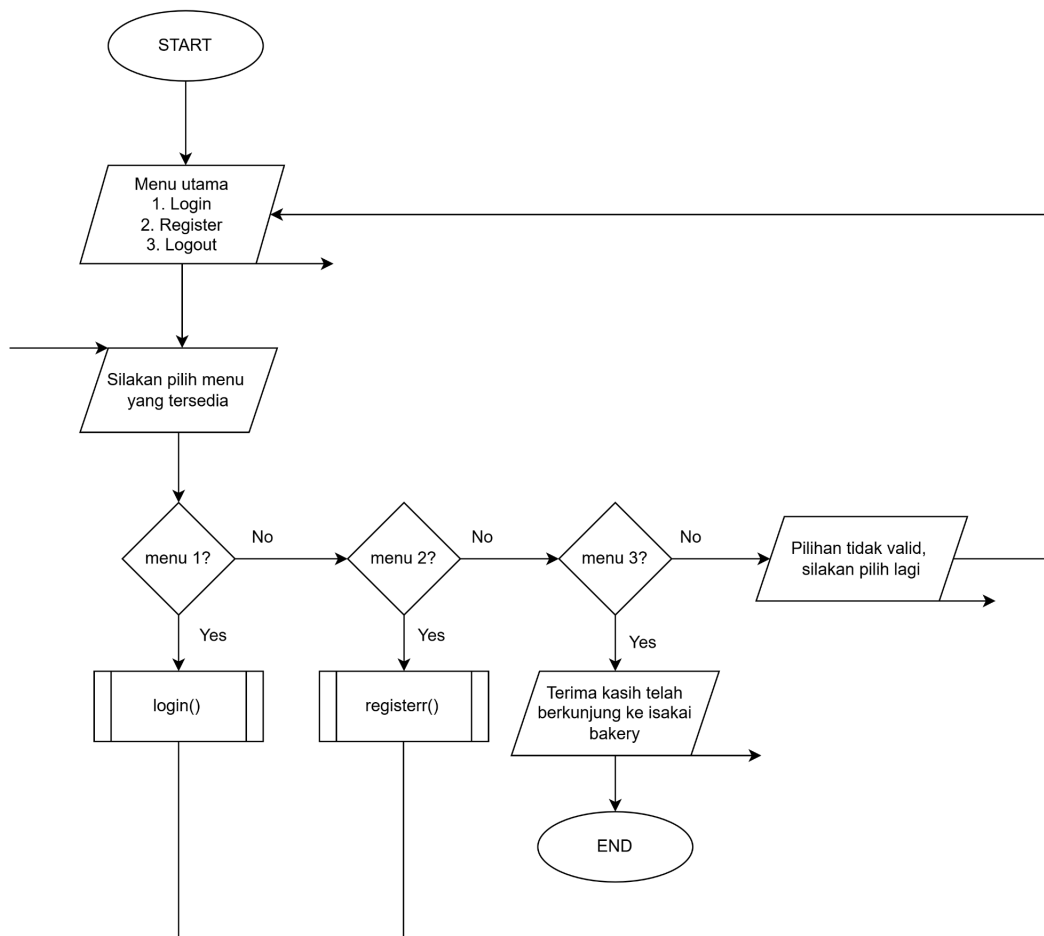
LAPORAN PRAKTIKUM
POST TEST 7
ALGORITMA PEMROGRAMAN LANJUT



Disusun oleh:
Syarafina Nur Amalia (2509106016)
Kelas A'25

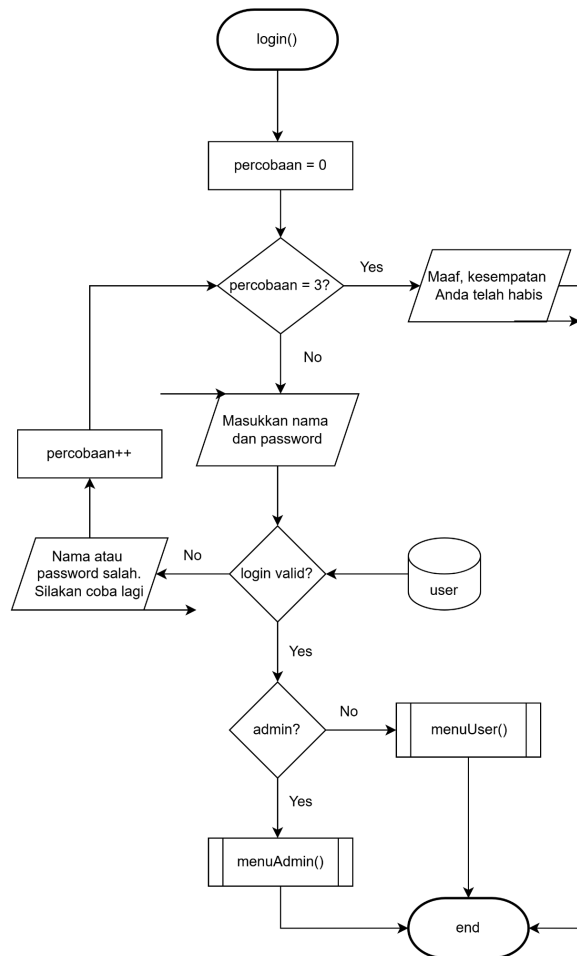
PROGRAM STUDI INFORMATIKA
UNIVERSITAS MULAWARMAN
SAMARINDA
2025

1. Flowchart



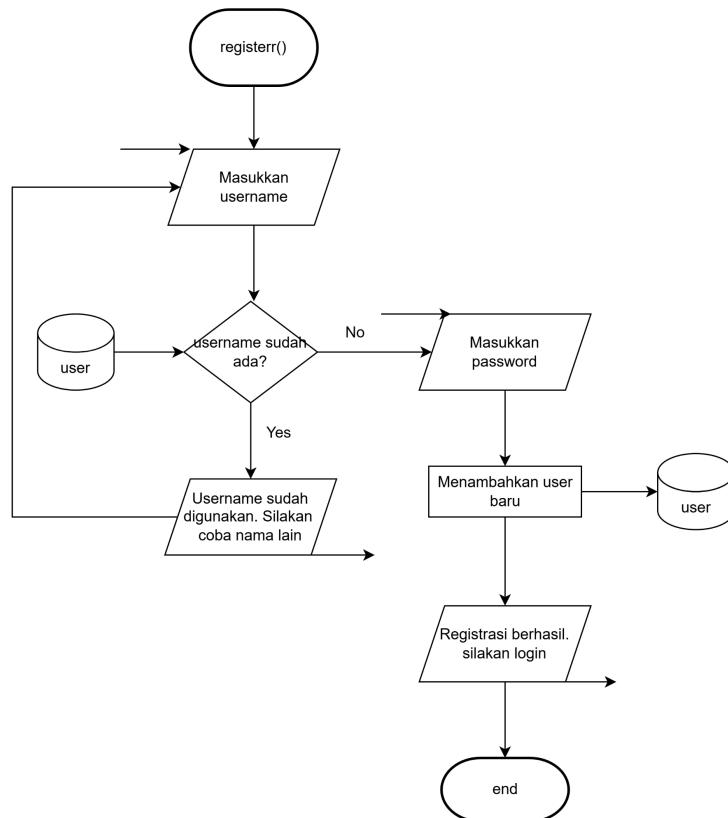
1.1 flowchart menu utama

Pada awal program, terdapat 3 menu utama, yaitu *login*, *register*, dan *logout*. Jika *user* memilih *login*, maka *user* akan masuk ke dalam pre defined *login*. Pada menu kedua, *user* akan masuk ke menu pre defined *register*, jika *user* belum terdaftar. Lalu menu ketiga akan membuat program berhenti. Jika *user* memilih selain ketiga menu yang tersedia, maka program akan menampilkan tulisan “Pilihan tidak valid, silakan pilih lagi.”



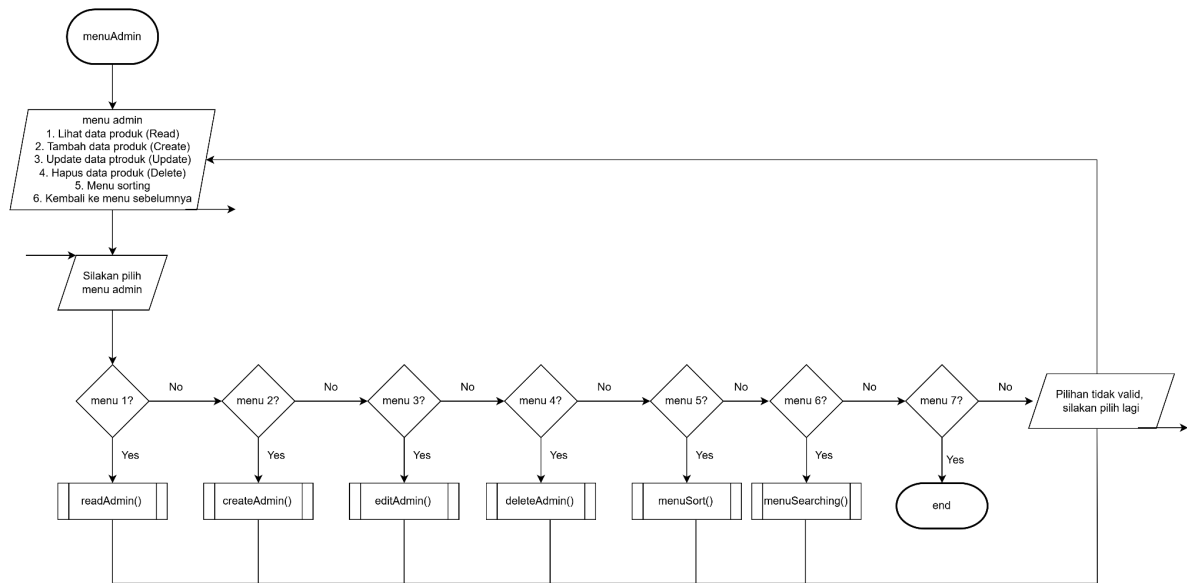
1.2 flowchart menu login

Ketika *user* masuk ke dalam menu pre defined *login*, *user* diberikan 3 kali kesempatan untuk melakukan proses *login*, jika gagal dalam 3 kali percobaan tersebut, maka user tidak bisa mengakses program dan akan tampil *output* “Maaf, kesempatan Anda telah habis.” Ketika sudah berhasil *login*, program akan mengecek apakah *user* merupakan admin atau bukan. Jika admin, maka akan masuk ke dalam pre defined *menuAdmin()* dan jika tidak, maka akan masuk ke dalam pre defined *menuUser()*



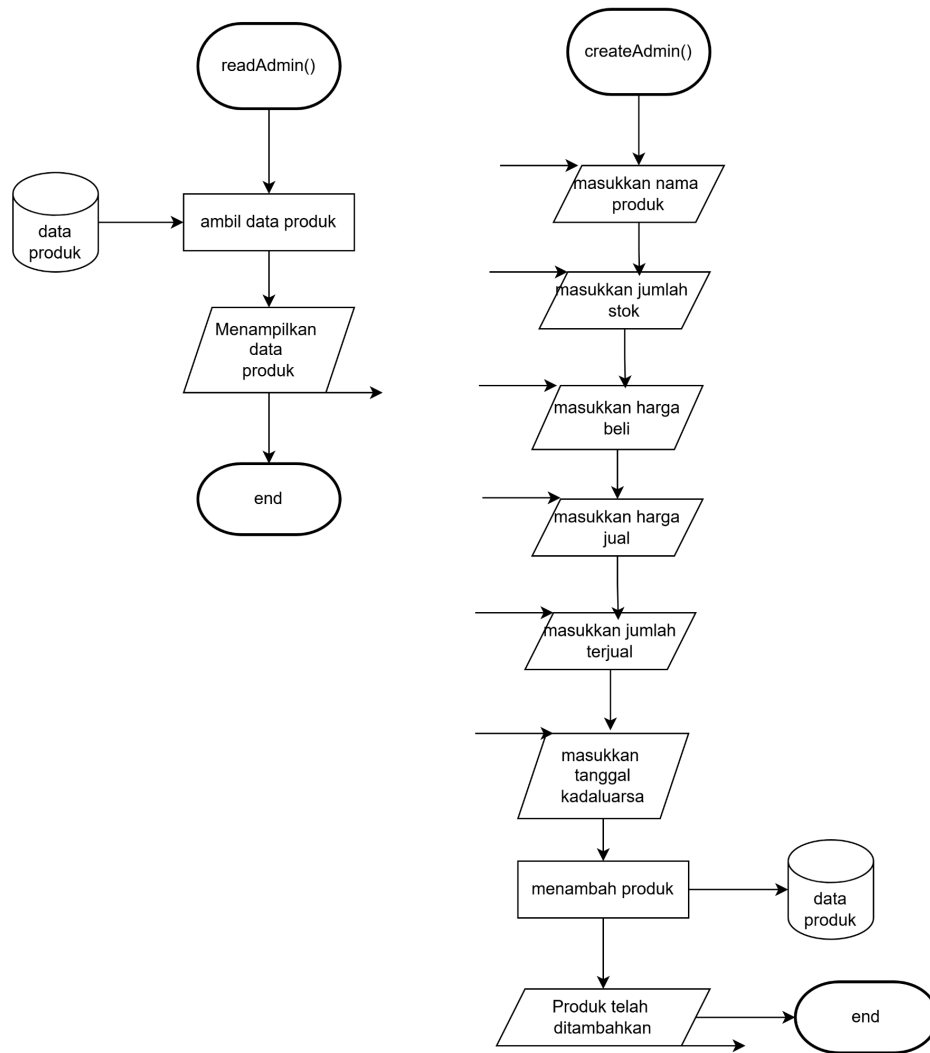
1.3 flowchart menu registrasi

Jika *user* tidak memiliki akun, maka ia bisa melakukan registrasi dengan memilih menu kedua dan program akan meminta *user* untuk memasukkan *username* yang belum terpakai, jika sudah terpakai, akan muncul output “Username sudah digunakan. Silakan coba nama lain.” Jika *username* yang dibuat belum terdaftar, maka program akan meminta *user* untuk memasukkan *password* dan membuat akun baru.

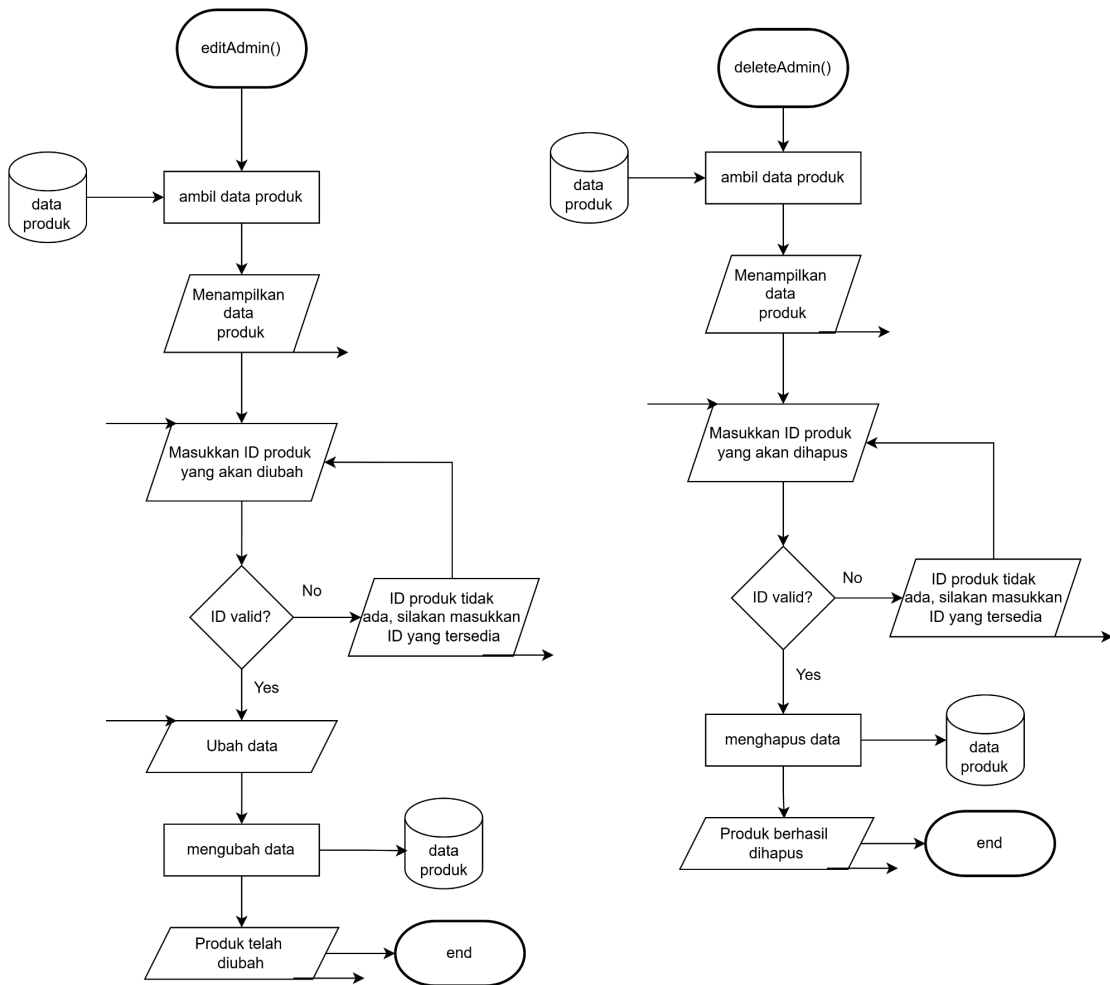


1.4 flowchart menu admin

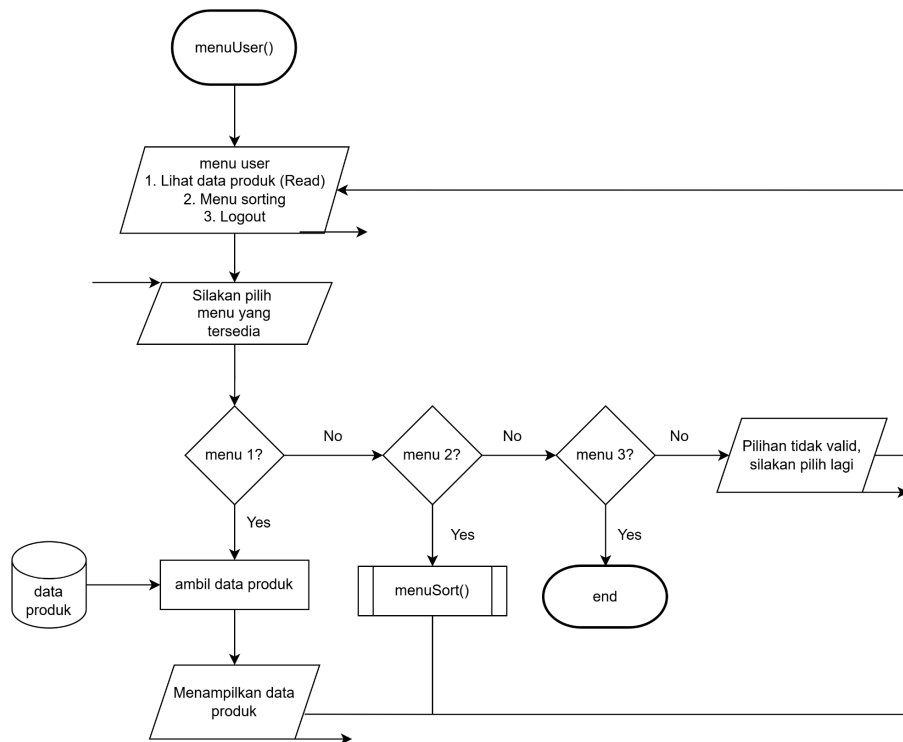
Jika *user* sudah login dan merupakan admin, maka akan muncul 7 menu admin, yaitu *read*, *create*, *update*, *delete*, menu sort, menu searching, dan *logout*.



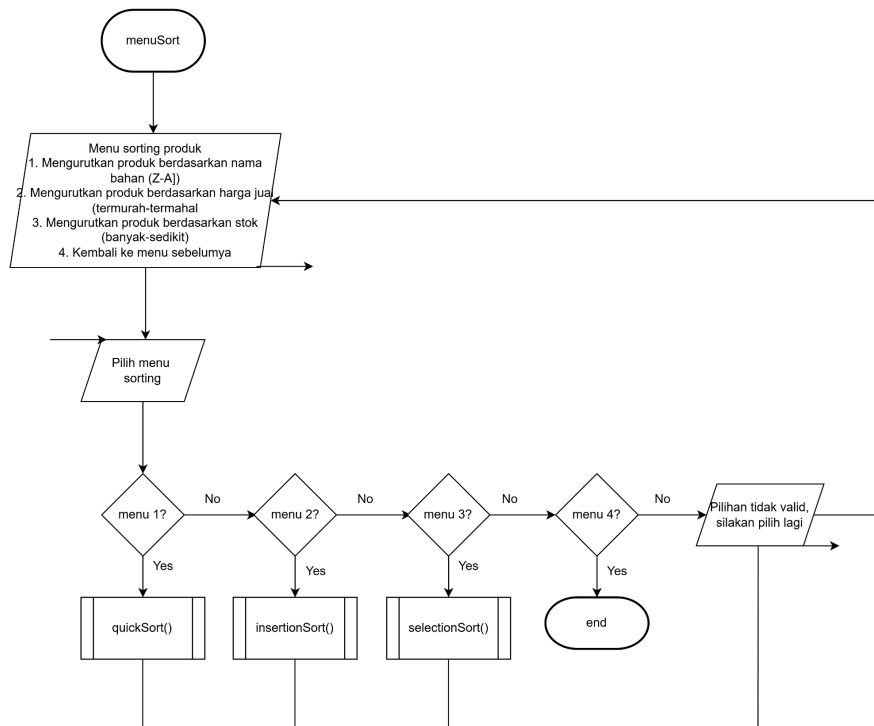
1.5 flowchart menu admin read dan create



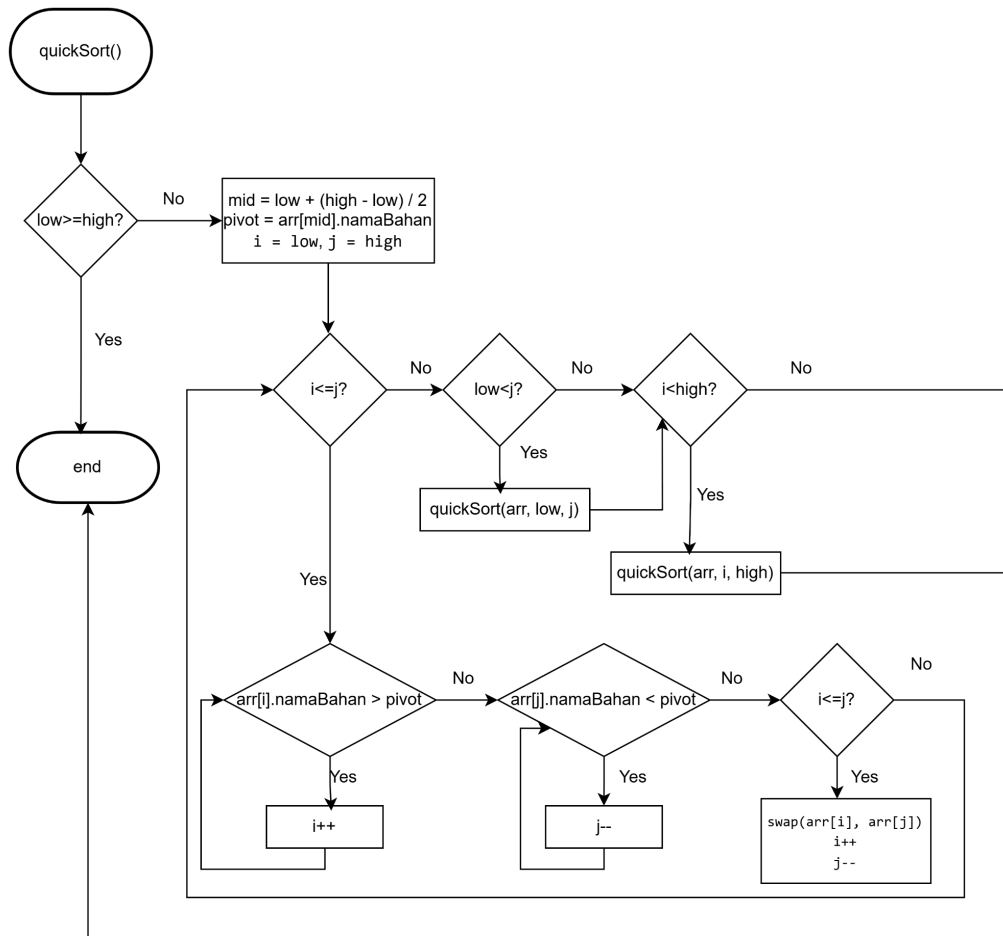
1.6 flowchart menu admin edit dan delete



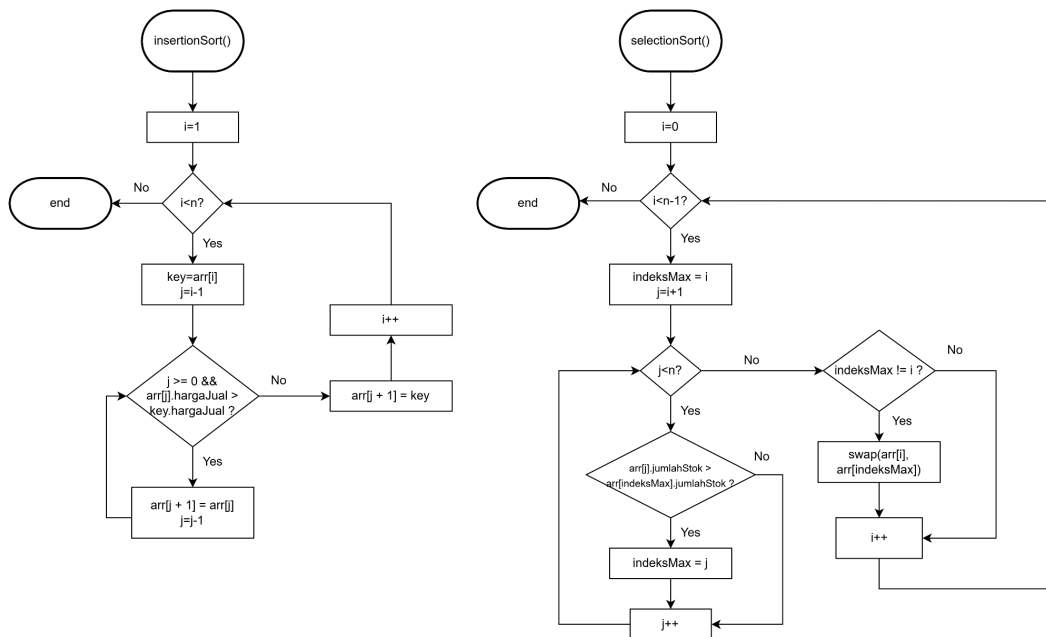
1.7 flowchart menu user



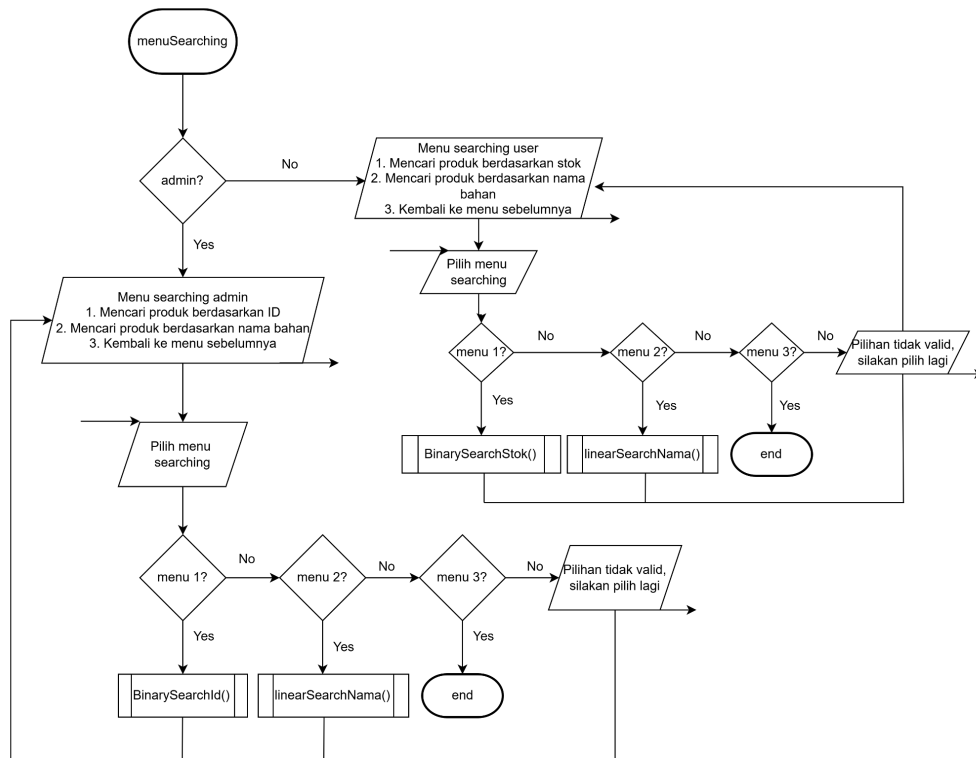
1.8 flowchart menu sort



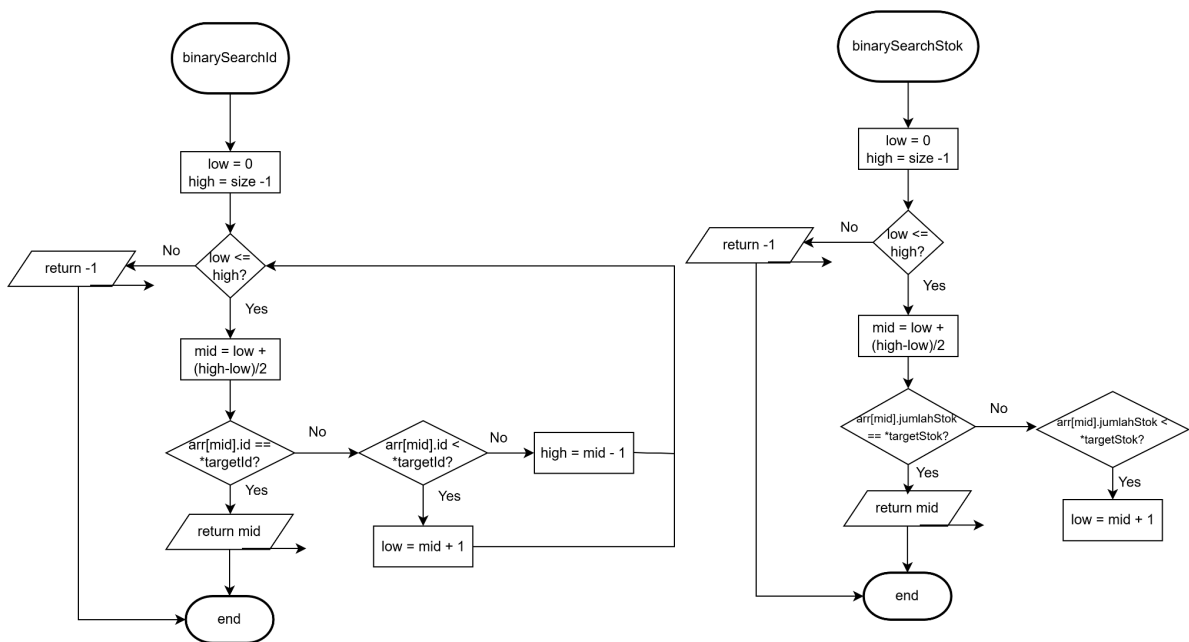
1.9 flowchart quick sort



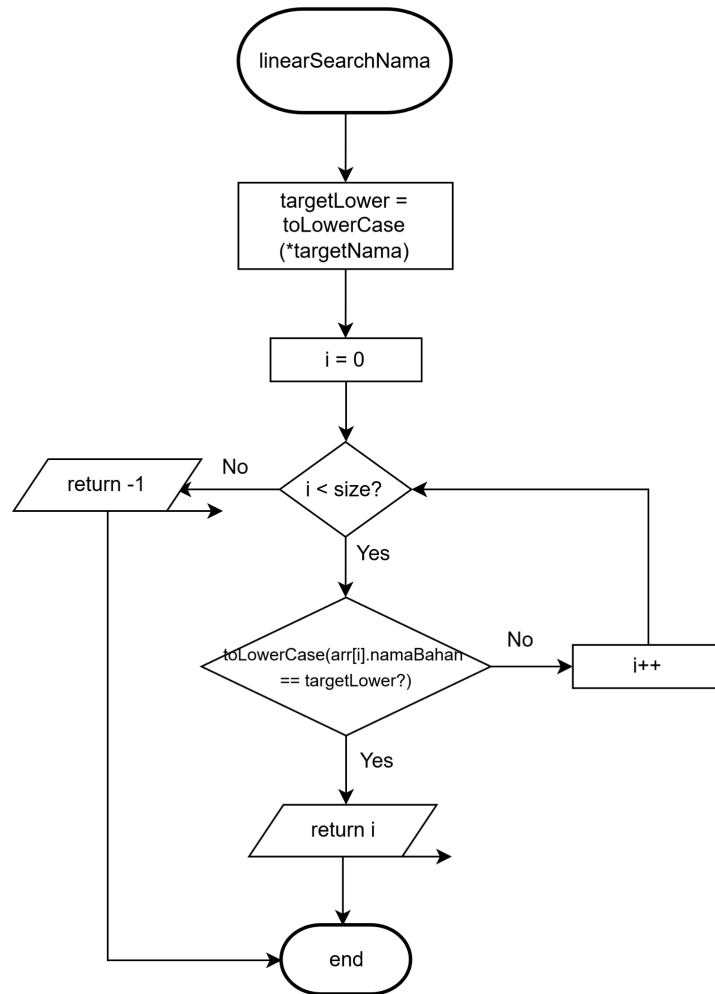
1.10 flowchart insertion dan selection sort



1.11 flowchart menu searching



1.12 flowchart binary search id dan stok



1.13 flowchart linear search nama bahan

2. Deskripsi Singkat Program

Program ini dibuat untuk mendata produk dari sebuah toko bahan kue dengan beberapa fitur, antara lain menambah, mengubah, menghapus, dan melihat produk yang tersedia. Program ini juga dilengkapi dengan sistem *login* dan registrasi, serta *role* sebagai admin atau *user* biasa, dimana kedua role tersebut memiliki aksesibilitas yang berbeda.

3. Source Code

```

struct kadaluarsa {
    int tanggal;
    int bulan;
    int tahun;
};

struct user {
    string usn;

```

```

    string pass;
    bool isAdmin;
};

struct produk {
    int id;
    string namaBahan;
    int jumlahStok;
    int hargaBeli;
    int hargaJual;
    int jumlahTerjual;
    kadaluarsa kadaluarsa; // nested struct
};

```

3.1 Source code struct

Code di atas merupakan *struct*, dimana saya menggunakannya untuk menyimpan data pengguna yang berisi *usn*, *pass*, dan *role* apakah admin atau bukan. Saya juga menggunakan *nested struct* pada variabel *kadaluarsa*, dimana ia menyimpan tanggal, bulan, dan tahun, yang dipanggil di dalam *struct* produk.

```

system("cls");

```

3.2 Source code clear terminal

Code di atas digunakan untuk membersihkan terminal pada saat program berjalan.

```

bool login(user users[], int jumlahUser, user &userAktif, int percobaan) {
    system("cls");
    header("LOGIN");

    string usn;
    cout << "Masukkan username : ";
    getline(cin, usn);
    string pass = password("Masukkan password : ");

    for (int i = 0; i < jumlahUser; i++) {
        if (users[i].usn == usn && users[i].pass == pass) {
            userAktif = users[i];
            cout << "\nLogin berhasil, selamat datang di Isekai Bakery " <<
users[i].usn << endl;
            return true;
        }
    }

    if (percobaan - 1 == 0) {
        cout << "\nMaaf, kesempatan Anda telah habis" << endl;
        return false;
    }
}

```

```

}

cout << "\nSayang sekali, login gagal" << endl;
cout << "Sisa percobaan Anda adalah: " << percobaan - 1;
cout << "\nTekan enter untuk mencoba lagi...";
cin.get();

return login(users, jumlahUser, userAktif, percobaan - 1);

```

3.3 Source code menu login

Code di atas merupakan menu *login* menggunakan fungsi rekursif yang dibatasi dengan jumlah percobaan tertentu melalui parameter percobaan. Jika sisa percobaan sudah mencapai 0, maka proses *login* akan dihentikan dan mengembalikan nilai *false*. Code di atas juga merupakan salah satu penerapan konsep *pass by reference* menggunakan tanda & pada parameter *userAktif*, sehingga data *user* yang berhasil *login* dapat tersimpan dan terhubung langsung ke variabel aslinya di program utama.

```

void registerr(user users[], int *jumlahUser, int maxUser) {
    system("cls");
    header("Form Registrasi");

    if (*jumlahUser >= maxUser) {
        cout << "Kapasitas user penuh" << endl;
        return;
    }

    string usnBaru;
    while (true) {
        cout << "Masukkan username : ";
        getline(cin, usnBaru);

        if (usnBaru.empty()) {
            cout << "Username tidak boleh kosong\n";
            continue;
        }

        bool isDuplicate = false;
        for (int i = 0; i < *jumlahUser; i++) {
            if (users[i].usn == usnBaru) {
                isDuplicate = true;
                break;
            }
        }
    }
}

```

```

        if (isDuplicate) {
            cout << "Username sudah digunakan. Silakan coba nama lain.\n";
        } else {
            users[*jumlahUser].usn = usnBaru;
            break;
        }
    }

    while (true) {
        string passBaru = password("Password : ");
        if (passBaru.empty()) {
            cout << "Password tidak boleh kosong\n";
        } else {
            users[*jumlahUser].pass = passBaru;
            break;
        }
    }

    users[*jumlahUser].isAdmin = false;

    (*jumlahUser)++;
    cout << "\nRegistrasi berhasil, silakan login" << endl;

```

3.4 Source code registrasi

Code di atas merupakan menu registrasi yang berfungsi untuk menambahkan *user* baru ke dalam program dengan melakukan pengecekan kapasitas maksimal serta validasi data masukan. Code di atas merupakan salah satu penerapan konsep *pointer* dengan menggunakan *dereference operator* (*) pada parameter *jumlahUser*. Melalui penggunaan *dereference* tersebut, pembaruan data seperti penambahan total akun melalui sintaks *(jumlahUser)++* akan secara langsung memodifikasi nilai variabel aslinya pada memori utama program.

```

void readAdmin(produk produkList[], int jumlahProduk, bool isAdmin) {
    Table tbl;
    if (isAdmin) {
        tbl.add_row({"No", "ID", "Nama Bahan", "Stok", "Harga Beli", "Harga Jual", "Terjual", "Kadaluarsa"});
    } else {
        tbl.add_row({"No", "Produk", "Stok", "Harga", "Terjual", "Kadaluarsa"});
    }

    tbl.row(0).format().font_style({FontStyle::bold}).font_align(FontAlign::center);

    for (int i = 0; i < jumlahProduk; i++) {
        string tgl = to_string(produkList[i].kadaluarsa.tanggal) + "/" +

```

```

to_string(produkList[i].kadaluarsa.bulan) + "/" +
to_string(produkList[i].kadaluarsa.tahun);
    if (isAdmin) {
        tbl.add_row({ to_string(i + 1), to_string(produkList[i].id),
produkList[i].namaBahan, to_string(produkList[i].jumlahStok), "Rp" +
to_string(produkList[i].hargaBeli), "Rp" + to_string(produkList[i].hargaJual),
to_string(produkList[i].jumlahTerjual), tgl });
    } else {
        tbl.add_row({ to_string(i + 1), produkList[i].namaBahan,
to_string(produkList[i].jumlahStok), "Rp" + to_string(produkList[i].hargaJual),
to_string(produkList[i].jumlahTerjual), tgl });
    }
}

tbl.column(0).format().width(5).font_align(FontAlign::center);

if (isAdmin) {
    tbl.column(1).format().width(5).font_align(FontAlign::center);
    tbl.column(2).format().width(18).font_align(FontAlign::center);
    tbl.column(3).format().width(8).font_align(FontAlign::center);
    tbl.column(4).format().width(12).font_align(FontAlign::center);
    tbl.column(5).format().width(12).font_align(FontAlign::center);
    tbl.column(6).format().width(10).font_align(FontAlign::center);
    tbl.column(7).format().width(12).font_align(FontAlign::center);
} else {
    tbl.column(1).format().width(18).font_align(FontAlign::center);
    tbl.column(2).format().width(8).font_align(FontAlign::center);
    tbl.column(3).format().width(12).font_align(FontAlign::center);
    tbl.column(4).format().width(10).font_align(FontAlign::center);
    tbl.column(5).format().width(12).font_align(FontAlign::center);
}
cout << tbl << endl;
}

```

3.5 Source code menu read

Code di atas untuk menampilkan data produk. Data akan ditampilkan dalam bentuk tabel. Terdapat perbedaan antara menu *read* admin dan *user*. Jika *login* sebagai *user*, maka tabel *ID* tidak akan diperlihatkan.

```

void createAdmin(produk produkList[], int *jumlahProduk, int maxProduk) {
    system("cls");
    header("Tambah Produk");

    if (*jumlahProduk >= maxProduk) {
        cout << "Kapasitas produk penuh" << endl;
        return;
    }
}

```

```

    produk *produkBaru = &produkList[*jumlahProduk];

    int maxId = 0;
    for (int i = 0; i < *jumlahProduk; i++) {
        if (produkList[i].id > maxId) maxId = produkList[i].id;
    }
    produkBaru->id = maxId + 1;

    while (true) {
        try {
            cout << "Nama produk : ";
            getline(cin, produkBaru->namaBahan);

            if (produkBaru->namaBahan.empty()) {
                throw invalid_argument("Nama produk tidak boleh kosong");
            }

            bool isHuruf = true;
            for (char c : produkBaru->namaBahan) {
                if (!((c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') || c
== ' ')) {
                    isHuruf = false; break;
                }
            }
            if (!isHuruf) throw invalid_argument("Silakan masukkan huruf
saja");

            break;
        } catch(const exception& e) {
            cout << "Error: " << e.what() << endl;
        }
    }

    while (true) {
        try {
            cout << "Jumlah stok : ";
            if (cin.peek() == '\n')
cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cin >> produkBaru->jumlahStok;
            if (cin.fail()) {
                cin.clear(); cin.ignore(numeric_limits<streamsize>::max(),
'\n');

                throw invalid_argument("Input harus berupa angka");
            }
            if (produkBaru->jumlahStok < 0) throw runtime_error("Angka harus
positif");
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            break;
        }
    }
}

```

```

        } catch(const exception& e) { cout << "Error: " << e.what() << endl;
    }
}

while (true) {
    try {
        cout << "Harga Beli (Rp) : ";
        cin >> produkBaru->hargaBeli;
        if (cin.fail()) {
            cin.clear(); cin.ignore(numeric_limits<streamsize>::max(),
'\n');
            throw invalid_argument("Input harus berupa angka");
        }
        if (produkBaru->hargaBeli < 0) throw runtime_error("Harga harus
positif");
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        break;
    } catch(const exception& e) { cout << "Error: " << e.what() << endl;
}
}

while (true) {
    try {
        cout << "Harga Jual (Rp) : ";
        cin >> produkBaru->hargaJual;
        if (cin.fail()) {
            cin.clear(); cin.ignore(numeric_limits<streamsize>::max(),
'\n');
            throw invalid_argument("Input harus berupa angka");
        }
        if (produkBaru->hargaJual < 0) throw runtime_error("Harga harus
positif");
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        break;
    } catch(const exception& e) { cout << "Error: " << e.what() << endl;
}
}

while (true) {
    try {
        cout << "Jumlah Terjual : ";
        cin >> produkBaru->jumlahTerjual;
        if (cin.fail()) {
            cin.clear(); cin.ignore(numeric_limits<streamsize>::max(),
'\n');
            throw invalid_argument("Input harus berupa angka");
        }
        if (produkBaru->jumlahTerjual < 0) throw runtime_error("Angka

```

```

harus positif");
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    break;
} catch(const exception& e) { cout << "Error: " << e.what() << endl;
}
}

cout << "\n--- Tanggal Kadaluarsa ---" << endl;
while (true) {
    try {
        cout << "Tanggal : ";
        cin >> produkBaru->kadaluarsa.tanggal;
        if (cin.fail()) {
            cin.clear(); cin.ignore(numeric_limits<streamsize>::max(),
'\n');
            throw invalid_argument("Input harus berupa angka");
        }
        if (produkBaru->kadaluarsa.tanggal <= 0 ||
produkBaru->kadaluarsa.tanggal > 31) throw out_of_range("Tanggal harus
antara 1-31");
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        break;
    } catch(const exception& e) { cout << "Error: " << e.what() << endl;
}
}

while (true) {
    try {
        cout << "Bulan : ";
        cin >> produkBaru->kadaluarsa.bulan;
        if (cin.fail()) {
            cin.clear(); cin.ignore(numeric_limits<streamsize>::max(),
'\n');
            throw invalid_argument("Input harus berupa angka");
        }
        if (produkBaru->kadaluarsa.bulan <= 0 ||
produkBaru->kadaluarsa.bulan > 12) throw out_of_range("Bulan harus antara
1-12");
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        break;
    } catch(const exception& e) { cout << "Error: " << e.what() << endl;
}
}

while (true) {
    try {
        cout << "Tahun : ";
        cin >> produkBaru->kadaluarsa.tahun;

```

```

        if (cin.fail()) {
            cin.clear(); cin.ignore(numeric_limits<streamsize>::max(),
'\n');
            throw invalid_argument("Input harus berupa angka");
        }
        if (produkBaru->kadaluarsa.tahun <= 0) throw out_of_range("Tahun
tidak valid");
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        break;
    } catch(const exception& e) { cout << "Error: " << e.what() << endl;
}
}

(*jumlahProduk)++;
cout << "\nProduk telah ditambahkan" << endl;
}

```

3.6 Source code menu create admin

Code di atas merupakan menu untuk menambahkan data produk baru ke dalam program, yang dilengkapi dengan pembuatan ID otomatis (*auto-increment*) serta penyamaran password. Code di atas juga menerapkan konsep *dereference operator* (*) pada parameter jumlahProduk untuk memperbarui total data, serta *pointer* pada *struct*. Melalui variabel *pointer* produkBaru, program memanfaatkan *arrow operator* (->) untuk mengakses dan memodifikasi atribut-atribut *struct* produk secara langsung di dalam memori.

```

void editAdmin(produk produkList[], int jumlahProduk) {
    system("cls");
    header("Edit Data Produk");
    if (jumlahProduk == 0) { cout << "Belum ada produk untuk diubah." <<
endl; return; }

    readAdmin(produkList, jumlahProduk, true);
    int target, idx = -1;
    insertionSortId(produkList, jumlahProduk);

    while (true) {
        try {
            cout << "\nMasukkan ID produk yang akan diubah : ";
            if (cin.peek() == '\n')
cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cin >> target;
            if (cin.fail()) {
                cin.clear(); cin.ignore(numeric_limits<streamsize>::max(),
'\n');

```

```

        throw invalid_argument("Input harus berupa angka");
    }
    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    idx = binarySearchId(produkList, jumlahProduk, target);
    if (idx == -1) throw out_of_range("ID produk tidak ditemukan");
    break;
} catch(const exception& e) { cout << "Error: " << e.what() << endl;
}

}

produk *produkEdit = &produkList[idx];

while (true) {
    try {
        cout << "Nama produk [" << produkEdit->namaBahan << "] : ";
        string namaBaru;
        getline(cin, namaBaru);
        if (namaBaru.empty()) break;

        bool isHuruf = true;
        for (char c : namaBaru) {
            if (!((c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') || c
== ' ')) { isHuruf = false; break; }
        }
        if (!isHuruf) throw invalid_argument("Silakan masukkan huruf
saja");

        produkEdit->namaBahan = namaBaru;
        break;
    } catch(const exception& e) { cout << "Error: " << e.what() << endl;
}

}

while (true) {
    try {
        cout << "Jumlah stok [" << produkEdit->jumlahStok << "] : ";
        if (cin.peek() == '\n') {
            cin.ignore(numeric_limits<streamsize>::max(), '\n'); break; }
        int stokBaru;
        cin >> stokBaru;
        if (cin.fail()) {
            cin.clear(); cin.ignore(numeric_limits<streamsize>::max(),
'\n');

            throw invalid_argument("Input harus berupa angka");
        }
        if (stokBaru < 0) throw runtime_error("Stok harus positif");
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
}

```

```

        produkEdit->jumlahStok = stokBaru; break;
    } catch(const exception& e) { cout << "Error: " << e.what() << endl;
}
    }

    while (true) {
        try {
            cout << "Harga Beli (Rp) [" << produkEdit->hargaBeli << "]" : ";
            if (cin.peek() == '\n') {
cin.ignore(numeric_limits<streamsize>::max(), '\n'); break; }
            int hargaBeliBaru;
            cin >> hargaBeliBaru;
            if (cin.fail()) {
                cin.clear(); cin.ignore(numeric_limits<streamsize>::max(),
'\n');
                throw invalid_argument("Input harus berupa angka");
            }
            if (hargaBeliBaru < 0) throw runtime_error("Harga harus
positif");
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            produkEdit->hargaBeli = hargaBeliBaru; break;
        } catch(const exception& e) { cout << "Error: " << e.what() << endl;
    }
}

    while (true) {
        try {
            cout << "Harga Jual (Rp) [" << produkEdit->hargaJual << "]" : ";
            if (cin.peek() == '\n') {
cin.ignore(numeric_limits<streamsize>::max(), '\n'); break; }
            int hargaJualBaru;
            cin >> hargaJualBaru;
            if (cin.fail()) {
                cin.clear(); cin.ignore(numeric_limits<streamsize>::max(),
'\n');
                throw invalid_argument("Input harus berupa angka");
            }
            if (hargaJualBaru < 0) throw runtime_error("Harga harus
positif");
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            produkEdit->hargaJual = hargaJualBaru; break;
        } catch(const exception& e) { cout << "Error: " << e.what() << endl;
    }
}

    while (true) {
        try {
            cout << "Jumlah Terjual [" << produkEdit->jumlahTerjual << "]" :

```

```

";
        if (cin.peek() == '\n') {
cin.ignore(numeric_limits<streamsize>::max(), '\n'); break; }
        int terjualBaru;
        cin >> terjualBaru;
        if (cin.fail()) {
            cin.clear(); cin.ignore(numeric_limits<streamsize>::max(),
'\n');
            throw invalid_argument("Input harus berupa angka");
        }
        if (terjualBaru < 0) throw runtime_error("Jumlah terjual harus
positif");
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        produkEdit->jumlahTerjual = terjualBaru; break;
    } catch(const exception& e) { cout << "Error: " << e.what() << endl;
}
    }

    cout << "\n--- Tanggal Kadaluarsa ---" << endl;
    while (true) {
        try {
            cout << "Tanggal [" << produkEdit->kadaluarsa.tanggal << "]" : ";
            if (cin.peek() == '\n') {
cin.ignore(numeric_limits<streamsize>::max(), '\n'); break; }
            int tglBaru;
            cin >> tglBaru;
            if (cin.fail()) {
                cin.clear(); cin.ignore(numeric_limits<streamsize>::max(),
'\n');
                throw invalid_argument("Input harus berupa angka");
            }
            if (tglBaru <= 0 || tglBaru > 31) throw out_of_range("Tanggal
harus antara 1-31");
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            produkEdit->kadaluarsa.tanggal = tglBaru; break;
        } catch(const exception& e) { cout << "Error: " << e.what() << endl;
    }
}

    while (true) {
        try {
            cout << "Bulan [" << produkEdit->kadaluarsa.bulan << "]" : ";
            if (cin.peek() == '\n') {
cin.ignore(numeric_limits<streamsize>::max(), '\n'); break; }
            int blnBaru;
            cin >> blnBaru;
            if (cin.fail()) {
                cin.clear(); cin.ignore(numeric_limits<streamsize>::max(),

```

```

'\n');
        throw invalid_argument("Input harus berupa angka");
    }
    if (blnBaru <= 0 || blnBaru > 12) throw out_of_range("Bulan
harus antara 1-12");
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    produkEdit->kadaluarsa.bulan = blnBaru; break;
} catch(const exception& e) { cout << "Error: " << e.what() << endl;
}
}

while (true) {
    try {
        cout << "Tahun [" << produkEdit->kadaluarsa.tahun << "] : ";
        if (cin.peek() == '\n') {
            cin.ignore(numeric_limits<streamsize>::max(), '\n'); break; }
        int thnBaru;
        cin >> thnBaru;
        if (cin.fail()) {
            cin.clear(); cin.ignore(numeric_limits<streamsize>::max(),
'\n');
            throw invalid_argument("Input harus berupa angka");
        }
        if (thnBaru <= 0) throw out_of_range("Tahun tidak valid");
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        produkEdit->kadaluarsa.tahun = thnBaru; break;
    } catch(const exception& e) { cout << "Error: " << e.what() << endl;
}
}

    cout << "\nData berhasil diedit" << endl;
}
}

```

3.7 Source code menu update admin

Code di atas berfungsi untuk *update* dengan memasukkan *ID* produk yang ingin diubah. Jika ada data produk yang tidak ingin diubah, dapat menekan *enter* untuk lanjut ke bagian selanjutnya. code di atas juga menerapkan pointer pada *struct*. Dengan mendeklarasikan variabel *pointer produkEdit* yang merujuk pada indeks *array* produk yang dipilih, program dapat langsung mengakses dan memodifikasi atribut-atribut *struct* di dalam memori menggunakan *Arrow Operator* (*->*).

```

void deleteAdmin(produk produkList[], int *jumlahProduk) {
    system("cls");
    header("Hapus Data Produk");
}

```

```

        if (*jumlahProduk == 0) { cout << "Belum ada produk untuk dihapus." << endl; return; }
        readAdmin(produkList, *jumlahProduk, true);

        int targetId, idx = -1;
        insertionSortId(produkList, *jumlahProduk);

        while (true) {
            try {
                cout << "\nMasukkan ID produk yang akan dihapus : ";
                if (cin.peek() == '\n')
                    cin.ignore(numeric_limits<streamsize>::max(), '\n');
                cin >> targetId;
                if (cin.fail()) {
                    cin.clear(); cin.ignore(numeric_limits<streamsize>::max(), '\n');
                    throw invalid_argument("Input harus berupa angka");
                }
                cin.ignore(numeric_limits<streamsize>::max(), '\n');

                idx = binarySearchId(produkList, *jumlahProduk, targetId);
                if (idx == -1) throw out_of_range("ID produk tidak ditemukan");
                break;
            } catch(const exception& e) { cout << "Error: " << e.what() << endl; }
        }

        for (int i = idx; i < *jumlahProduk - 1; i++) {
            produkList[i] = produkList[i + 1];
        }
        (*jumlahProduk)--;
        cout << "\nProduk berhasil dihapus" << endl;
    }
}

```

3.8 Source code menu delete admin

Code di atas berfungsi untuk menghapus produk dengan memilih *ID* produk yang ingin dihapus. Program akan menampilkan semua produk yang ada sebelum *user* memilih *ID* agar memudahkan *user* untuk memilih *ID* yang ingin dihapus. Proses penghapusan datanya dilakukan dengan cara menggeser elemen *array* untuk menimpa indeks data yang dihapus. *Code* di atas juga menerapkan konsep *pointer* dengan menggunakan *dereference operator* (*) pada parameter *jumlahProduk*. Melalui penggunaan *dereference* tersebut, pengurangan total jumlah produk melalui *(*jumlahProduk)--* akan secara langsung memodifikasi nilai variabel aslinya pada memori utama.

```

void menuUser(produk produkList[], int jumlahProduk, bool &sudahLogin) {
    int menuUserChoice;
    do {
        system("cls");
        header("Menu User");
        Table menuUserTbl;
        menuUserTbl.add_row({"No", "Menu"});
        menuUserTbl.add_row({"1", "Lihat produk"});
        menuUserTbl.add_row({"2", "Menu Sorting"});
        menuUserTbl.add_row({"3", "Menu Searching"});
        menuUserTbl.add_row({"4", "Kembali ke menu sebelumnya"});
        menuUserTbl.row(0).format().font_align(FontAlign::center);

        menuUserTbl.column(0).format().width(6).font_align(FontAlign::center);
        menuUserTbl.column(1).format().width(30);
        cout << menuUserTbl << endl;

        while (true) {
            try {
                cout << "Silakan pilih menu yang tersedia: ";
                if (cin.peek() == '\n')
                    cin.ignore(numeric_limits<streamsize>::max(), '\n');
                cin >> menuUserChoice;
                if (cin.fail()) {
                    cin.clear();
                    cin.ignore(numeric_limits<streamsize>::max(), '\n');
                    throw invalid_argument("Input harus berupa angka");
                }
                cin.ignore(numeric_limits<streamsize>::max(), '\n');
                break;
            } catch(const exception& e) { cout << "Error: " << e.what() <<
endl; }
        }

        switch (menuUserChoice) {
            case 1:
                system("cls"); header("Daftar Produk (User)");
                if (jumlahProduk == 0) cout << "Belum ada data produk." <<
endl;

                else readAdmin(produkList, jumlahProduk, false);
                break;
            case 2: menuSorting(produkList, jumlahProduk, false); break;
            case 3: menuSearching(produkList, jumlahProduk, false); break;
            case 4: sudahLogin = false; break;
            default: cout << "Pilihan tidak valid, silakan pilih lagi" <<
endl;
        }
        if (menuUserChoice != 4 && menuUserChoice != 2 && menuUserChoice !=

```

```

3) {
    cout << "\nTekan enter untuk melanjutkan..."; cin.get();
}
} while (menuUserChoice != 4 && sudahLogin);
}

```

3.9 Source code menu user biasa

Code di atas berfungsi untuk menampilkan menu yang dimiliki oleh *user* biasa, dimana hanya terdapat 3 menu, yaitu melihat data, menu sorting, dan juga *logout*.

Jika *user* memilih menu 1, maka akan tampil semua data produk yang berisi No, nama produk, harga, terjual, stok, dan kadaluarsa. Jika memilih menu 2, maka *user* akan diarahkan ke menu sorting data. Jika memilih menu 3, maka *user* akan diarahkan kembali ke menu utama.

```

int main() {
    const int maxUser = 100;
    const int maxProduk = 100;

    user users[maxUser];
    int jumlahUser = 0;

    produk produkList[maxProduk];
    int jumlahProduk = 0;

    user userAktif;
    bool sudahLogin = false;

    users[0].usn = "senku"; users[0].pass = "1234"; users[0].isAdmin =
false;
    users[1].usn = "fina"; users[1].pass = "016"; users[1].isAdmin = true;
    jumlahUser = 2;

    produkList[0] = {1, "Ragi", 50, 1000, 1500, 100, {15, 12, 2026}};
    produkList[1] = {2, "Pewarna", 30, 8000, 12000, 80, {10, 8, 2026}};
    produkList[2] = {3, "Gula", 40, 8000, 12000, 80, {26, 10, 2026}};
    jumlahProduk = 3;

    int menuUtama;

    do {
        system("cls");
        cout << "\n===== ISEKAI BAKERY =====\n" << endl;
        Table menuUtamaTbl;
    }
}

```

```

        menuUtamaTbl.addRow({"No", "Menu"});
        menuUtamaTbl.addRow({"1", "Login"});
        menuUtamaTbl.addRow({"2", "Register"});
        menuUtamaTbl.addRow({"3", "Logout"});
        menuUtamaTbl.row(0).format().font_align(FontAlign::center);

menuUtamaTbl.column(0).format().width(6).font_align(FontAlign::center);
menuUtamaTbl.column(1).format().width(20);
cout << menuUtamaTbl << endl;

while (true) {
    try {
        cout << "Silakan pilih menu yang tersedia : ";
        if (cin.peek() == '\n')
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
        cin >> menuUtama;
        if (cin.fail()) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            throw invalid_argument("Silakan masukkan angka saja.");
        }
        if (menuUtama < 1 || menuUtama > 3) throw
out_of_range("Pilihan di luar dari senarai menu!");
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        break;
    } catch (const exception& e) { cout << "Error: " << e.what() <<
endl << endl; }
}

switch (menuUtama) {
    case 1: {
        bool statusLogin = login(users, jumlahUser, userAktif, 3);
        if (statusLogin) {
            sudahLogin = true;
            cout << "\nTekan enter untuk melanjutkan..."; cin.get();
            if (userAktif.isAdmin) menuAdmin(produkList,
jumlahProduk, maxProduk, sudahLogin);
            else menuUser(produkList, jumlahProduk, sudahLogin);
        } else {
            cout << "==== Terima kasih telah berkunjung ke Isekai
Bakery =====" << endl; return 0;
        }
        break;
    }
    case 2:
        registerr(users, &jumlahUser, maxUser);
        cout << "\nTekan enter untuk melanjutkan..."; cin.get();
        break;
}

```

```

        case 3:
            cout << "===== Terima kasih telah berkunjung ke Isekai Bakery =====" << endl;
            break;
        }
    } while (menuUtama != 3);

    return 0;
}

```

3.10 Source code main()

Code di atas merupakan fungsi main() yang mengatur jalannya program. Program menggunakan *do-while* untuk menampilkan tabel menu utama dengan 3 pilihan menu. Program juga menggunakan *while(true)* agar program tidak *error* saat *user* memasukkan inputan kosong maupun huruf. Selain itu, *code* di atas juga menerapkan *address of operator* (&) secara langsung, yaitu pada saat pemanggilan fungsi `registerr(users, &jumlahUser, maxUser)`, di mana program mengirimkan alamat memori dari variabel `jumlahUser` agar nilainya dapat diperbarui langsung dari dalam fungsi tersebut.

```

void menuSorting(produk produkList[], int jumlahProduk, bool isAdmin) {
    if (jumlahProduk == 0) { cout << "Belum ada data produk." << endl;
return; }
    int pilihan;
    do {
        system("cls");
        header("Menu Sorting Produk");
        Table tbl;
        tbl.add_row({"No", "Menu Sorting"});
        tbl.add_row({"1", "Mengurutkan Produk Berdasarkan Nama Bahan (Z-A)"});
        tbl.add_row({"2", "Mengurutkan Produk Berdasarkan Harga Jual (Termurah-Termahal)"});
        tbl.add_row({"3", "Mengurutkan Produk Berdasarkan Stok (Banyak-Sedikit)"});
        tbl.add_row({"4", "Kembali ke menu sebelumnya"});
        tbl.row(0).format().font_align(FontAlign::center);
        tbl.column(0).format().width(5).font_align(FontAlign::center);
        cout << tbl << endl;

        while (true) {
            try {
                cout << "Pilih menu sorting: ";
                if (cin.peek() == '\n')

```

```

cin.ignore(numeric_limits<streamsize>::max(), '\n');
    cin >> pilihan;
    if (cin.fail()) {
        cin.clear();
cin.ignore(numeric_limits<streamsize>::max(), '\n');
        throw invalid_argument("Input harus berupa angka");
    }
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    break;
} catch(const exception& e) { cout << "Error: " << e.what() <<
endl; }
}

if (pilihan == 1) {
    quickSort(produkList, 0, jumlahProduk - 1);
    system("cls"); cout << "\nData Berdasarkan Nama Bahan
(Z-A)\n\n";
    readAdmin(produkList, jumlahProduk, isAdmin);
    cout << "\nTekan enter untuk melanjutkan..."; cin.get();
} else if (pilihan == 2) {
    insertionSort(produkList, jumlahProduk);
    system("cls"); cout << "\nData Berdasarkan Harga Jual
(Termurah-Termahal)\n\n";
    readAdmin(produkList, jumlahProduk, isAdmin);
    cout << "\nTekan enter untuk melanjutkan..."; cin.get();
} else if (pilihan == 3) {
    selectionSort(produkList, jumlahProduk);
    system("cls"); cout << "\nData Berdasarkan Jumlah Stok
(Banyak-Sedikit)\n\n";
    readAdmin(produkList, jumlahProduk, isAdmin);
    cout << "\nTekan enter untuk melanjutkan..."; cin.get();
} else if (pilihan != 4) {
    cout << "Pilihan tidak valid, silakan pilih lagi\nTekan
enter..."; cin.get();
}
} while (pilihan != 4);
}

```

3.11 Source code menu sorting

Code di atas merupakan menu sorting dengan mekanisme *Error Handling* menggunakan *try-catch*. Jika *cin.fail()* mendeteksi *input* yang salah, program akan melempar (*throw*) *invalid argument* dan mengulang permintaan *input*. Lalu, program menggunakan *if-else* di dalam perulangan *do-while* untuk mengeksekusi algoritma *sorting* spesifik (Quick Sort, Insertion Sort, atau Selection Sort) sesuai pilihan pengguna.

```

void menuSearching(produk produkList[], int jumlahProduk, bool isAdmin) {
    if (jumlahProduk == 0) { cout << "Belum ada data produk." << endl;
return; }
    int pilihan;
    do {
        system("cls");
        header("Menu Searching Produk");
        Table tbl;
        tbl.add_row({"No", "Menu Searching"});
        if (isAdmin) tbl.add_row({"1", "Mencari produk berdasarkan ID"});
        else tbl.add_row({"1", "Mencari produk berdasarkan jumlah stok"});
        tbl.add_row({"2", "Mencari produk berdasarkan nama bahan"});
        tbl.add_row({"3", "Kembali ke menu sebelumnya"});
        tbl.row(0).format().font_align(FontAlign::center);
        tbl.column(0).format().width(5).font_align(FontAlign::center);
        cout << tbl << endl;

        while (true) {
            try {
                cout << "Pilih menu searching: ";
                if (cin.peek() == '\n')
cin.ignore(numeric_limits<streamsize>::max(), '\n');
                cin >> pilihan;
                if (cin.fail()) {
                    cin.clear();
cin.ignore(numeric_limits<streamsize>::max(), '\n');
                    throw invalid_argument("Input harus berupa angka");
                }
                cin.ignore(numeric_limits<streamsize>::max(), '\n');
                break;
            } catch(const exception& e) { cout << "Error: " << e.what() <<
endl; }
        }

        if (pilihan == 1) {
            if (isAdmin) {
                int idCari;
                while (true) {
                    try {
                        cout << "\nMasukkan ID Produk yang dicari: ";
                        cin >> idCari;
                        if (cin.fail()) {
                            cin.clear();
cin.ignore(numeric_limits<streamsize>::max(), '\n');
                            throw invalid_argument("Input harus berupa
angka");
                        }
                    }
                    if (idCari <= 0) throw runtime_error("ID harus angka

```

```

positif");
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        break;
    } catch(const exception& e) { cout << "Error: " <<
e.what() << endl; }
    }
    system("cls");
    header("Hasil Pencarian Berdasarkan ID");
    insertionSortId(produkList, jumlahProduk);
    int idx = binarySearchId(produkList, jumlahProduk, idCari);
    if (idx != -1) cetakSatuProduk(produkList[idx], isAdmin);
    else cout << "Produk dengan ID " << idCari << " tidak
ditemukan." << endl;
    } else {
        int stokCari;
        while (true) {
            try {
                cout << "\nMasukkan Jumlah Stok yang dicari: ";
                cin >> stokCari;
                if (cin.fail()) {
                    cin.clear();
cin.ignore(numeric_limits<streamsize>::max(), '\n');
                    throw invalid_argument("Input harus berupa
angka");
                }
                if (stokCari < 0) throw runtime_error("Stok tidak
boleh negatif");
                cin.ignore(numeric_limits<streamsize>::max(), '\n');
                break;
            } catch(const exception& e) { cout << "Error: " <<
e.what() << endl; }
        }
        system("cls");
        header("Hasil Pencarian Berdasarkan Stok");
        insertionSortByStok(produkList, jumlahProduk);
        int idx = binarySearchStok(produkList, jumlahProduk,
stokCari);
        if (idx != -1) cetakSatuProduk(produkList[idx], isAdmin);
        else cout << "Produk dengan stok " << stokCari << " tidak
ditemukan." << endl;
    }
    cout << "\nTekan enter untuk melanjutkan..."; cin.get();
} else if (pilihan == 2) {
    string namaCari;
    while (true) {
        try {
            cout << "\nMasukkan Nama Bahan yang dicari: ";
            getline(cin, namaCari);

```

```

        if (namaCari.empty()) throw invalid_argument("Silakan
masukkan nama produk");
        bool isHuruf = true;
        for (char c : namaCari) {
            if (!((c >= 'a' && c <= 'z') || (c >= 'A' && c <=
'Z') || c == ' ')) { isHuruf = false; break; }
        }
        if (!isHuruf) throw invalid_argument("Silakan masukkan
huruf saja");
        break;
    } catch(const exception& e) { cout << "Error: " << e.what()
<< endl; }
    }
    system("cls");
    header("Hasil Pencarian Berdasarkan Nama");
    int idx = linearSearchNama(produkList, jumlahProduk, namaCari);
    if (idx != -1) cetakSatuProduk(produkList[idx], isAdmin);
    else cout << "Produk '" << namaCari << "' tidak ditemukan." <<
endl;
    cout << "\nTekan enter untuk melanjutkan..."; cin.get();
    } else if (pilihan != 3) {
        cout << "Pilihan tidak valid, silakan pilih lagi\nTekan
enter..."; cin.get();
    }
    } while (pilihan != 3);
}

```

3.12 Source code menu searching

Code di atas merupakan menu *searching* dengan menggunakan *error handling* try-catch. Jika *cin.fail()* mendeteksi *input* yang salah, program akan melempar (*throw*) *invalid argument* atau *runtime error* dan mengulang permintaan *input*. Lalu, program menggunakan *if-else* di dalam perulangan *do-while* untuk mengeksekusi algoritma *searching* spesifik (*Binary Search* atau *Linear Search*) sesuai pilihan pengguna.

4. Hasil Output

```
===== ISEKAI BAKERY =====

+-----+-----+
| No |      Menu      |
+-----+-----+
|  1  | Login          |
+-----+-----+
|  2  | Register       |
+-----+-----+
|  3  | Logout         |
+-----+-----+
Silakan pilih menu yang tersedia : █
```

Gambar 4.1 Menu utama

```
===== LOGIN =====

Masukkan username : fina
Masukkan password : ***

Login berhasil, selamat datang di Isekai Bakery fina
Tekan enter untuk melanjutkan...█
```

Gambar 4.2 Login sebagai admin

```
===== Menu Admin =====

+-----+-----+
| No |      Menu      |
+-----+-----+
|  1  | Lihat data produk (Read) |
+-----+-----+
|  2  | Tambah produk baru (Create) |
+-----+-----+
|  3  | Update data produk (Update) |
+-----+-----+
|  4  | Hapus data produk (Delete) |
+-----+-----+
|  5  | Menu Sorting      |
+-----+-----+
|  6  | Menu Searching    |
+-----+-----+
|  7  | Kembali ke menu sebelumnya |
+-----+-----+
Silakan pilih menu admin: █
```

Gambar 4.3 Menu admin

```
===== Data Produk (Admin) =====
```

No	ID	Nama Bahan	Stok	Harga Beli	Harga Jual	Terjual	Kadaluarsa
1	1	Ragi	50	Rp1000	Rp1500	100	15/12/2026
2	2	Pewarna	30	Rp8000	Rp12000	80	10/8/2026
3	3	Gula	40	Rp8000	Rp12000	80	26/10/2026

Tekan enter untuk melanjutkan...

Gambar 4.4 Menu read admin

```
===== Tambah Produk =====

Nama produk : Tepung terigu
Jumlah stok : 60
Harga Beli (Rp) : 23000
Harga Jual (Rp) : 25000
Jumlah Terjual : 34

--- Tanggal Kadaluarsa ---
Tanggal : 22
Bulan : 8
Tahun : 2026

Produk telah ditambahkan

Tekan enter untuk melanjutkan...
```

Gambar 4.5 Menu create admin

```
===== Tambah Produk =====

Nama produk :
Nama produk tidak boleh kosong!
Nama produk : Vanilli
Jumlah stok :
Silakan masukkan stok dalam bentuk angka
Jumlah stok : g
Tolong masukkan dalam bentuk angka
Jumlah stok : 67
Harga Beli (Rp) : j
Tolong masukkan dalam bentuk angka
Harga Beli (Rp) :
Silakan masukkan harga beli dalam bentuk angka
Harga Beli (Rp) : 12000
Harga Jual (Rp) :
Silakan masukkan harga jual dalam bentuk angka
Harga Jual (Rp) : j
Tolong masukkan dalam bentuk angka
Harga Jual (Rp) : 13000
Jumlah Terjual : 45

--- Tanggal Kadaluarsa ---
Tanggal :
Silakan masukkan tanggal dalam bentuk angka
Tanggal : j
Tolong masukkan dalam bentuk angka
Tanggal : 22
Bulan : 9
Tahun : 2026

Produk telah ditambahkan

Tekan enter untuk melanjutkan...|
```

Gambar 4.6 Menu create beserta error handling

```

Masukkan ID produk yang akan diubah : 3
Nama produk [Gula]:
Jumlah stok [40]: 39
Harga Beli (Rp) [8000]:
Harga Jual (Rp) [12000]:
Jumlah Terjual [80]:

--- Tanggal Kadaluarsa ---
tanggal [26]:
Bulan [10]:
Tahun [2026]:

Data berhasil diedit

Tekan enter untuk melanjutkan...

```

Gambar 4.7 Menu admin update data produk

```

===== Hapus Data Produk =====

+-----+-----+-----+-----+-----+-----+-----+-----+
| No | ID | Nama Bahan | Stok | Harga Beli | Harga Jual | Terjual | Kadaluarsa |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | 1 | Ragi | 50 | Rp1000 | Rp1500 | 100 | 15/12/2026 |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 2 | 2 | Pewarna | 30 | Rp8000 | Rp12000 | 80 | 10/8/2026 |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 3 | 3 | Gula | 40 | Rp8000 | Rp12000 | 80 | 26/10/2026 |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 4 | 4 | Tepung terigu | 60 | Rp23000 | Rp25000 | 34 | 22/8/2026 |
+-----+-----+-----+-----+-----+-----+-----+-----+

Masukkan ID produk yang akan dihapus : 2

Produk berhasil dihapus

Tekan enter untuk melanjutkan...

```

Gambar 4.8 Menu admin delete data produk

```

===== Data Produk (Admin) =====

+-----+-----+-----+-----+-----+-----+-----+-----+
| No | ID | Nama Bahan | Stok | Harga Beli | Harga Jual | Terjual | Kadaluarsa |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | 1 | Ragi | 50 | Rp1000 | Rp1500 | 100 | 15/12/2026 |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 2 | 3 | Gula | 40 | Rp8000 | Rp12000 | 80 | 26/10/2026 |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 3 | 4 | Tepung terigu | 60 | Rp23000 | Rp25000 | 34 | 22/8/2026 |
+-----+-----+-----+-----+-----+-----+-----+-----+

Tekan enter untuk melanjutkan...

```

Gambar 4.9 Reset nomor produk yang didelete

```
===== Form Registrasi =====  
  
Masukkan username : fdf  
Password : ***  
  
Registrasi berhasil, silakan login  
  
Tekan enter untuk melanjutkan...|
```

Gambar 4.10 Menu registrasi

```
===== LOGIN =====  
  
Masukkan username : fdf  
Masukkan password : ***  
  
Login berhasil, selamat datang di Isekai Bakery fdf  
  
Tekan enter untuk melanjutkan...|
```

Gambar 4.11 Login menggunakan akun yang baru registrasi

```
===== LOGIN =====  
  
Masukkan username : ffff  
Masukkan password : ***  
  
Sayang sekali, login gagal  
Sisa percobaan Anda adalah: 2  
Tekan enter untuk mencoba lagi...|
```

Gambar 4.12 Gagal login 1 kali

```
===== LOGIN =====  
  
Masukkan username : ggg  
Masukkan password : ****  
  
Sayang sekali, login gagal  
Sisa percobaan Anda adalah: 1  
Tekan enter untuk mencoba lagi...|
```

Gambar 4.13 Gagal login 2 kali

```

===== LOGIN =====

Masukkan username : dfd
Masukkan password : ****

Maaf, kesempatan Anda telah habis
===== Terima kasih telah berkunjung ke Isekai Bakery =====

```

Gambar 4.14 Gagal login 3 kali

```

===== Menu User =====

+-----+-----+
| No | Menu |
+-----+-----+
| 1 | Lihat produk |
+-----+-----+
| 2 | Menu Sorting |
+-----+-----+
| 3 | Menu Searching |
+-----+-----+
| 4 | Kembali ke menu sebelumnya |
+-----+-----+
Silakan pilih menu yang tersedia: 

```

Gambar 4.15 Menu user

```

===== Daftar Produk (User) =====

+-----+-----+-----+-----+-----+-----+
| No | Produk | Stok | Harga | Terjual | Kadaluarsa |
+-----+-----+-----+-----+-----+-----+
| 1 | Ragi | 50 | Rp1500 | 100 | 15/12/2026 |
+-----+-----+-----+-----+-----+-----+
| 2 | Pewarna | 30 | Rp12000 | 80 | 10/8/2026 |
+-----+-----+-----+-----+-----+-----+
| 3 | Gula | 40 | Rp12000 | 80 | 26/10/2026 |
+-----+-----+-----+-----+-----+-----+

Tekan enter untuk melanjutkan...

```

Gambar 4.16 Menu read user biasa

```

===== Menu Sorting Produk =====

+-----+-----+
| No | Menu Sorting |
+-----+-----+
| 1 | Mengurutkan Produk Berdasarkan Nama Bahan (Z-A) |
+-----+-----+
| 2 | Mengurutkan Produk Berdasarkan Harga Jual (Termurah-Termahal) |
+-----+-----+
| 3 | Mengurutkan Produk Berdasarkan Stok (Banyak-Sedikit) |
+-----+-----+
| 4 | Kembali ke menu sebelumnya |
+-----+-----+
Pilih menu sorting: 

```

Gambar 4.17 Menu sorting

Data Berdasarkan Nama Bahan (Z-A)

No	ID	Nama Bahan	Stok	Harga Beli	Harga Jual	Terjual	Kadaluarsa
1	1	Ragi	50	Rp1000	Rp1500	100	15/12/2026
2	2	Pewarna	30	Rp8000	Rp12000	80	10/8/2026
3	3	Gula	40	Rp8000	Rp12000	80	26/10/2026

Tekan enter untuk melanjutkan...

Gambar 4.18 Menu sorting secara descending menggunakan quick sort

Data Berdasarkan Harga Jual (Termurah-Termahal)

No	ID	Nama Bahan	Stok	Harga Beli	Harga Jual	Terjual	Kadaluarsa
1	1	Ragi	50	Rp1000	Rp1500	100	15/12/2026
2	2	Pewarna	30	Rp8000	Rp12000	80	10/8/2026
3	3	Gula	40	Rp8000	Rp12000	80	26/10/2026

Tekan enter untuk melanjutkan...

Gambar 4.19 Menu sorting secara ascending menggunakan insertion sort

Data Berdasarkan Jumlah Stok (Banyak-Sedikit)

No	ID	Nama Bahan	Stok	Harga Beli	Harga Jual	Terjual	Kadaluarsa
1	1	Ragi	50	Rp1000	Rp1500	100	15/12/2026
2	3	Gula	40	Rp8000	Rp12000	80	26/10/2026
3	2	Pewarna	30	Rp8000	Rp12000	80	10/8/2026

Tekan enter untuk melanjutkan...

Gambar 4.18 Menu sorting secara descending menggunakan selection sort

===== Menu Searching Produk =====

No	Menu Searching
1	Mencari produk berdasarkan jumlah stok
2	Mencari produk berdasarkan nama bahan
3	Kembali ke menu sebelumnya

Pilih menu searching: 1

Masukkan Jumlah Stok yang dicari: 20

Gambar 4.19 Menu searching jumlah stok menggunakan binary search

```

===== Menu Searching Produk =====

+-----+-----+
| No |           Menu Searching           |
+-----+-----+
| 1 | Mencari produk berdasarkan ID      |
+-----+-----+
| 2 | Mencari produk berdasarkan nama bahan |
+-----+-----+
| 3 | Kembali ke menu sebelumnya         |
+-----+-----+
Pilih menu searching: 2

Masukkan Nama Bahan yang dicari: 2

```

Gambar 4.20 Menu searching id menggunakan binary search

```

===== Menu Searching Produk =====

+-----+-----+
| No |           Menu Searching           |
+-----+-----+
| 1 | Mencari produk berdasarkan ID      |
+-----+-----+
| 2 | Mencari produk berdasarkan nama bahan |
+-----+-----+
| 3 | Kembali ke menu sebelumnya         |
+-----+-----+
Pilih menu searching: 2

Masukkan Nama Bahan yang dicari: ragi

```

Gambar 4.21 Menu searching nama bahan menggunakan linear search

5. Langkah-langkah GIT

5.1 GIT ADD

```
PS C:\Users\w180w\OneDrive\Documents\praktikum-apl\post-test\post-test-apl-7> git add .
```

Fungsi dari git add . adalah untuk menambahkan semua file yang ada di dalam directory.

5.2 GIT COMMIT

```
PS C:\Users\w180w\OneDrive\Documents\praktikum-apl\post-test\post-test-apl-7> git commit -m "last pt"
[master b714695] last pt
4 files changed, 978 insertions(+)
create mode 100644 post-test/post-test-apl-7/2509106016-SyarafinaNurAmalia-PT-7.cpp
create mode 100644 post-test/post-test-apl-7/2509106016-SyarafinaNurAmalia-PT-7.exe
create mode 100644 post-test/post-test-apl-7/auth.h
create mode 100644 post-test/post-test-apl-7/operasi.h
```

Fungsi dari git commit adalah untuk konfirmasi perubahan yang terjadi di repository, jadi bisa melihat riwayat perubahan yang dilakukan sebelumnya.

5.3 GIT Push

```
PS C:\Users\w180w\OneDrive\Documents\praktikum-apl\post-test\post-test-apl-7> git push -u origin master
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 8 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (8/8), 835.93 KiB | 5.07 MiB/s, done.
Total 8 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/haroldie666/praktikum-apl.git
 8929af0..b714695 master -> master
branch 'master' set up to track 'origin/master'.
```

Fungsi dari git push adalah untuk mengupload file yang berada di lokal ke github.